
EMIGA CODESYSTEM UPDATE MANAGER (ECUM) — DOCUMENTATION

Project: EMIGA Codesystem Release Manager

Version: 0.1 (Prototype)

Author:

Date: 2026-05-21

1. Project Overview and Purpose

The EMIGA Release Manager automates the end-to-end process of managing FHIR-CodeSystem changes for the rki.emiga.common repository. It provides a user-friendly SPA for authorized users to edit FSH content, validates edits, commits changes in an environment, runs SUSHI to generate FHIR JSON, builds packaged artifacts, and optionally publishes them to the Simplifier FHIR package registry.

Goals:

- Eliminate manual steps for FSH → SUSHI → Package → Simplifier
- Ensure repeatable, auditable, and versioned pipelines
- Maintain main repository untouched during prototype
- **Provide sandbox tooling, validation, CI templates, and documentation**

2. Diagram/Use case actors

- Authorized User : Edits FSH via SPA UI
- Backend API: Validates edits, creates branches, triggers pipeline
- CI Runner: Executes SUSHI, validation, packaging, and publishing

- Simplifier Registry: Receives FHIR packages

3. Scope & Deliverables

1. Specification Docs (requirements, UML sketches)
2. Sandbox CLI under tools/release-manager-cli (gitignored)
3. Backend API OpenAPI contract (GET/POST resources, POST /release/trigger)
4. UI Wireframe (SPA for FSH editing and validation review)
5. CI Pipeline Template (GitHub Actions / GitLab)
6. Validation Module (schema + semantic checks, JSON reports)
7. Packaging Module (version bump, npm .tgz build)
8. Simplifier Publish Adapter (dry-run + configurable credentials)
9. Unit & Integration Tests
10. README + Operator Runbook (**logs?**)

4. Inputs & Outputs

Inputs:

- Repository FSH and JSON resources
- User-provided FSH edits (UI)
- Configuration: registry credentials, semver policy, workspace paths

Outputs:

- Validation report (.json)
- Commit (sandbox) with SHA / PR URL
- Generated FHIR JSON artifacts
- Package (.tgz)

- Publication confirmation (Simplifier response)
- Logs and notifications (email)

5. Pipeline Steps (High Level)

1. UI: Edit FSH via form-based (code-display) perform client-side validation
2. Backend API: Authenticate, validate FHIR, check duplicates, create branch, respond with SHA
3. Trigger: webhook or repository_dispatch triggers CI
4. CI Runner:
 - a. Checkout repository
 - b. Validate resources (Firely/HAPI)
 - c. Combine FSH edits
 - d. Run SUSHI → generate FHIR JSON
 - e. Merge outputs, bump semver
 - f. Build package (.tgz) artifacts
 - g. Release notes (generated)
5. Publish: Upload to Simplifier (API/CLI), confirm publication, upload logs, notify user

6. Validation Rules & Acceptance Criteria

- FHIR validation via .net /java HAPI;
 - Codes/identifiers: enforce configurable formats
 - Automated suggestions with confidence <0.95 flagged for manual review
 - Success = Package published + confirmation returned + artifacts stored
-

7. Non-Functional Requirements

- Authentication
- Audit trail (user, timestamp, commit SHA) for prod
- Idempotent pipeline triggers
- Configurable timeouts and retry policies for external calls (prod)

8. Constraints

- No direct writes to main branch in prototype
- Credentials injected via CI secrets; never hardcoded
- Offline SUSHI runs supported locally and in CI

9. Suggested Tools & Libraries

- Backend: .NET 7, FastAPI / ASP.NET Core
- UI: React or Blazor SPA
- SUSHI: fsh-sushi (npm)
- Validation: Firely SDK / HAPI Validator
- CI: GitHub Actions / GitLab CI
- Packaging: .tgz
- Python libs (optional): pandas, rapidfuzz
- Logging / Notifications: webhook / email / CI artifacts

10. File Structure (Prototype)

docs/

release-manager.txt <- this document

release-manager-spec.md <- requirements + UML

tools/

release-manager-cli/ <- sandbox CLI, gitignored

openapi/

release-manager.yaml <- API contract stub

ci/

workflows/

release-pipeline.yml <- CI template

ui/

wireframes/

SPA-mockup.png

tests/

unit/

integration/

README.md <- operator instructions

runbook.md <- steps for dry-run, publish, rollback

11. Success Criteria

- Complete specification with UML diagrams
- Working sandbox CLI (combine → sushi → package → zip)
- CI workflow template ready to connect (no secrets)
- Backend API stub with OpenAPI example responses
- Operator runbook for maintainers to enable registry credentials and switch dry-run → publish

=====